

# Marketing Analytics

## Chapter 2 · Describing Data

Nils Wlömert

Vienna University of Economics and Business (WU)

October 2026

# Before we start

## Paper quiz on Chapter 1 (10 min)

- Individual, closed book
- Counts toward participation
- Questions look like the learning check at the end of last session

## Setup check

- Who completed the **R onboarding** checkpoint (report knitted to HTML)?
- Still stuck? → **setup clinic** now, during the quiz debrief — bring your laptop and the error message

# Today's session

## Part A — Getting to know the data

- 2.1 A first conversation with your data
- 2.2 Getting data into R
- 2.3 Variables and levels of measurement
- 2.4 Data handling

## Part B — Describing it well

- 2.5 Not trusting your data too early
- 2.6 Summarizing data
- 2.7 Visualizing data
- 2.8 From description to insight

**Laptops open:** today is a live-coding session. Code along in your RStudio Project.

## 2.1 A first conversation with your data

# Your first day in the data team

The head of A&R at a record label hands you a data set of tracks from the streaming charts:

*“What is going on in the charts?”*

- Which genres dominate?
- How concentrated is success?
- What distinguishes the hits from the rest?

**Driving question of this chapter:** *What is going on?* — the **descriptive** question from §1.3.

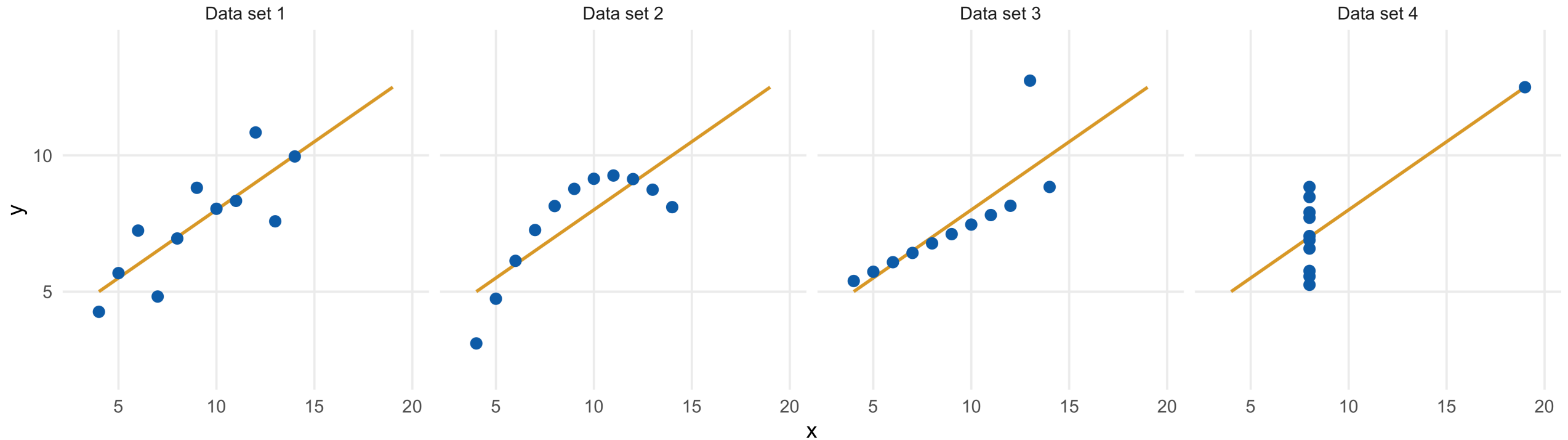
Before any model: **look at the data.**  
Description is where the problems are caught.

## Four data sets — one set of statistics

|            | Mean of x | Mean of y | SD of x | SD of y | Correlation |
|------------|-----------|-----------|---------|---------|-------------|
| Data set 1 | 9         | 7.5       | 3.32    | 2.03    | 0.82        |
| Data set 2 | 9         | 7.5       | 3.32    | 2.03    | 0.82        |
| Data set 3 | 9         | 7.5       | 3.32    | 2.03    | 0.82        |
| Data set 4 | 9         | 7.5       | 3.32    | 2.03    | 0.82        |

Same means, same standard deviations, same correlation. **Are these four data sets alike?**

# Summaries compress — pictures reveal



A noisy line, a perfect curve, a line with one outlier, a “relationship” created by a single point — **identical summary statistics**. Good description always uses **both** numbers and pictures.

Anscombe (1973), *The American Statistician* — built-in R data set [anscombe](#).

# The data: tracks in the streaming charts

~67,000 tracks that appeared in the Spotify charts, with 31 variables:

| Group                         | Variables                                                                                    |
|-------------------------------|----------------------------------------------------------------------------------------------|
| Identifiers                   | isrc, artist_id, artistName, trackName                                                       |
| Chart performance             | streams, weeks_in_charts, n_regions, top10                                                   |
| Audio characteristics (0–100) | danceability, energy, speechiness, instrumentalness, liveness, valence                       |
| Track properties              | tempo, song_length, song_age, explicit, release_date, genre, label                           |
| Other platforms & reach       | youtube_views, tiktok_counts, shazam_counts, n_playlists, artist-level followers & listeners |
| Expert judgment               | expert_rating (poor – fair – good – excellent – masterpiece)                                 |



## 2.2 Getting data into R

# Reading the data

```
1 library(tidyverse)
2
3 music_url <- "https://raw.githubusercontent.com/wu-rds/MA2026/main/data/music_data_fin.csv"
4 music_data <- read_csv2(music_url)
```

Why `read_csv2()` and not `read_csv()`? What do you expect to happen with `read_csv()`?

# An import can fail silently

```
1 wrong_import <- read_csv(music_url)
2 dim(wrong_import)
```

```
[1] 66780      1
```

| Convention    | Columns separated by | Decimal mark | Function                 |
|---------------|----------------------|--------------|--------------------------|
| International | ,                    | .            | <code>read_csv()</code>  |
| European      | ;                    | ,            | <code>read_csv2()</code> |

**No error ≠ correct.** Worse variants go unnoticed: `3,14` silently becoming `314`. Always inspect right after importing.

# A first look

```
1 music_raw |>
2   select(streams, danceability, genre, label, release_date, expert_rating) |>
3   glimpse()
```

Rows: 66,796

Columns: 6

```
$ streams      <dbl> 11944813, 8934097, 38835, 46766, 2930573, 72199738, 1041694, 44363...
$ danceability <dbl> 50.9, 35.3, 68.3, 70.4, 84.2, 35.2, 73.0, 55.6, 71.9, 34.6, 82.0, ...
$ genre       <chr> "other", "Pop", "Pop", "HipHop/Rap", "HipHop/Rap", "Rock", "HipHop...
$ label       <chr> "Independent", "Universal Music", "Universal Music", "Independent"...
$ release_date <date> 2016-06-20, 2018-06-14, 2018-04-26, 2020-03-31, 2020-02-07, 2004-...
$ expert_rating <chr> "excellent", "good", "good", "poor", "excellent", "fair", "poor", ...
```

```
1 summary(select(music_raw, streams, song_length))
```

| streams  |            | song_length |          |
|----------|------------|-------------|----------|
| Min. :   | 1003       | Min. :      | 0.01667  |
| 1st Qu.: | 55760      | 1st Qu.:    | 2.92925  |
| Median : | 333336     | Median :    | 3.33978  |
| Mean :   | 7314674    | Mean :      | 3.43116  |
| 3rd Qu.: | 2181086    | 3rd Qu.:    | 3.81987  |
| Max. :   | 2165692479 | Max. :      | 29.78383 |

# Other formats — same logic

| Format                         | Package          | Function                                               |
|--------------------------------|------------------|--------------------------------------------------------|
| CSV (international / European) | readr            | <code>read_csv()</code> / <code>read_csv2()</code>     |
| Other delimiters               | readr            | <code>read_delim("file.txt", delim = "\t")</code>      |
| Excel                          | readxl           | <code>read_excel("file.xlsx", sheet = "Sheet1")</code> |
| SPSS / Stata                   | haven            | <code>read_sav()</code> / <code>read_dta()</code>      |
| Qualtrics export               | qualtRics        | <code>read_survey()</code> → group project             |
| Web pages / APIs               | rvest / jsonlite | see the course book ( <i>optional</i> )                |

Files in your project folder are read by their **relative path**: `read_excel("data/survey.xlsx")` — no `setwd()` needed inside an RStudio Project.

# What is a row?

**Tidy data** (Wickham, 2014):

1. Each **variable** is a column
2. Each **observation** is a row
3. Each **value** is a cell

**Untidy:** one column per month — “month” is hidden in the column names

| track   | Jan | Feb | Mar |
|---------|-----|-----|-----|
| Track A | 120 | 135 | 150 |
| Track B | 80  | 70  | 65  |

Before any analysis, answer explicitly: **what does one row represent?** — the **unit of observation** (cf. “unit of analysis” in §1.8). Here: one row = one track. *Does that really hold?* → \$2.5

**Tidy:** one row per track and month

| track   | month | streams |
|---------|-------|---------|
| Track A | Jan   | 120     |
| Track A | Feb   | 135     |
| Track A | Mar   | 150     |
| Track B | Jan   | 80      |
| Track B | Feb   | 70      |
| Track B | Mar   | 65      |

## 2.3 Variables and levels of measurement

# Levels of measurement: categorical

## Nominal

- Numbers only serve as **labels** for identification and categorization
- They do not reflect the **amount** of a characteristic
- Two categories: **binary**
- Permissible operation: **counting**

E.g., starting numbers in a race



$7 \cdot 11 \cdot 3$

## Ordinal

- Numbers indicate the **relative position** (order)
- ...but not the **size of the differences**
- Also: **greater / less than**; percentile-based statistics such as **median** and **quartiles**

E.g., finishing order in the race



1st · 2nd · 3rd place



# Levels of measurement: continuous (metric)

## Interval

- **Differences** can be compared (equal intervals)
- ...but the **zero point is arbitrary**
- Ratios are **not** meaningful
- Also: range, mean, standard deviation

E.g., finishing clock time (start: 14:00)



14:07:06 · 14:14:12 · 14:15:12

## Ratio

- All properties of the scales above
- **Absolute zero point**
- Ratios are meaningful (“**twice as fast**”)
- All statistical techniques apply

E.g., time to finish (minutes)



7.1 · 14.2 · 15.2

# Not every number is a quantity

| Level    | Values express                  | Music data              | Meaningful    | Summary                    |
|----------|---------------------------------|-------------------------|---------------|----------------------------|
| Nominal  | Categories, no order            | genre, label, artist_id | = / ≠         | Frequencies, mode          |
| Ordinal  | Ordered categories              | expert_rating           | + < / >       | + median, quantiles        |
| Interval | Equal distances, arbitrary zero | release_year, °C        | + differences | + mean, SD                 |
| Ratio    | Equal distances, natural zero   | streams, song_length    | + ratios      | + coefficient of variation |

Nominal + ordinal = **categorical**; interval + ratio = **continuous**. The level decides which **summaries, charts** — and later **tests** — are meaningful. The mean `artist_id` is as meaningless as the mean phone number.

## Your turn: classify these (2 min)

Nominal, ordinal, interval, or ratio?

- a. Customer ID · (b) Satisfaction: “very dissatisfied” ... “very satisfied” · (c) Price in € · (d) Payment method · (e) Year of first purchase · (f) Purchased (yes / no)

# Telling R the level of measurement

```

1 music_data <- music_raw |>
2   mutate(
3     genre      = as.factor(genre),                # nominal
4     explicit   = factor(explicit, levels = c(0, 1),
5                          labels = c("not explicit", "explicit")), # nominal (binary)
6     artist_id  = as.character(artist_id),          # an ID is not a number
7     expert_rating = factor(expert_rating,
8                            levels = c("poor", "fair", "good", "excellent", "masterpiece"),
9                            ordered = TRUE)          # ordinal
10  )

```

```
1 table(music_data$expert_rating)
```

| poor | fair  | good  | excellent | masterpiece |
|------|-------|-------|-----------|-------------|
| 5356 | 13418 | 23243 | 23376     | 1403        |

As text, R would sort the ratings **alphabetically**; as an **ordered factor**, R knows that *good* lies between *fair* and *excellent*.

## 2.4 Data handling

# The verbs of data handling

| Verb                                  | Does                                  | Example question                   |
|---------------------------------------|---------------------------------------|------------------------------------|
| <code>filter()</code>                 | keeps <b>rows</b> meeting a condition | Pop tracks with > 10M streams?     |
| <code>select()</code>                 | keeps <b>columns</b>                  | only artist, title, streams        |
| <code>arrange()</code>                | <b>sorts</b> rows                     | the most-streamed tracks?          |
| <code>mutate()</code>                 | creates / changes <b>columns</b>      | streams in millions? release year? |
| <code>group_by() + summarise()</code> | condenses to <b>one row per group</b> | median streams per label?          |
| <code>count()</code>                  | counts rows per group                 | tracks per genre?                  |
| <code>left_join()</code>              | <b>combines</b> tables via a key      | add label information              |

Same logic everywhere: data frame in → **column names without quotes** → data frame out. Chained with the pipe `|>`: “*and then*”.

# Chaining steps with the pipe

```

1 music_data |>                                # take the music data, then
2   filter(genre == "Pop") |>                  # keep the pop tracks, then
3   arrange(desc(streams)) |>                 # sort by streams, then
4   select(artist, track, streams) |>         # keep three columns, then
5   head(4)                                   # show the first four rows

```

# A tibble: 4 × 3

|   | artist                            | track        | streams    |
|---|-----------------------------------|--------------|------------|
|   | <chr>                             | <chr>        | <dbl>      |
| 1 | Ed Sheeran                        | Shape of You | 2165692479 |
| 2 | Shawn Mendes feat. Camila Cabello | Señorita     | 1156206588 |
| 3 | XXXTENTACION                      | SAD!         | 1103693478 |
| 4 | Ed Sheeran                        | Perfect      | 1045189446 |

## “... per ...”: group\_by() + summarise()

```
1 music_data |>
2   group_by(label) |>
3   summarise(
4     n_tracks      = n(),
5     median_streams = median(streams),
6     share_top10 = scales::percent(mean(top10 == "yes"), accuracy = 0.1)
7   )
```

# A tibble: 4 × 4

|   | label           | n_tracks | median_streams | share_top10 |
|---|-----------------|----------|----------------|-------------|
|   | <fct>           | <int>    | <dbl>          | <chr>       |
| 1 | Independent     | 22570    | 238884         | 7.9%        |
| 2 | Sony Music      | 12390    | 355898.        | 11.3%       |
| 3 | Universal Music | 21632    | 434943         | 11.3%       |
| 4 | Warner Music    | 10204    | 347152.        | 10.2%       |

The workhorse of descriptive analysis: every question of the form “... per ...” has this shape.



# Combining tables: left\_join()

```
1 label_info <- tibble(  
2   label      = c("Universal Music", "Sony Music", "Warner Music", "Independent"),  
3   label_type = c("Major", "Major", "Major", "Independent")  
4 )  
5 music_data <- music_data |> left_join(label_info, by = "label")  
6 music_data |> count(label_type)
```

```
# A tibble: 2 × 2  
  label_type      n  
  <chr>      <int>  
1 Independent 22570  
2 Major      44226
```

Keeps all rows of the left table; no match → **NA**. Check the row count after every join — a non-unique key duplicates rows silently.

## Your turn (live coding, 8 min)

Using the verbs, answer:

1. How many **hip-hop** tracks reached the **top 10**?
2. Which **label** has the highest **median streams** among hip-hop tracks?
3. Create a variable `streams_mio` (streams in millions) and show the five **rock** tracks with the most streams.

## 2.5 Not trusting your data too early

# Five checks before you trust a data set

1. **Did the import work?** Rows, columns, types — §2.2
2. **Missing values** — including zeros that mean “unknown”
3. **Impossible values** — ranges, dates, units
4. **Duplicates** — is the identifier really unique?
5. **Extreme values** — errors or information?

...and one question beyond the checks: **how did the data come to be?**

## Check 2: zeros that mean “unknown”

```
1 sum(is.na(music_data))           # missing values: none at all?
```

```
[1] 0
```

```
1 mean(music_data$youtube_views == 0)   # share of tracks with 0 YouTube views
```

```
[1] 0.5796155
```

58% of charting tracks with **zero** YouTube views? Almost certainly “**no data**”, not “no views”. **Disguised missing values** (revenue 0 for never-contacted customers, age 0 in the CRM) drag every average down. Recode: `na_if(youtube_views, 0)` — and report how many are missing.

## Check 3: impossible values

```
1 music_data |>
2   summarise(first_release = min(release_date), last_release = max(release_date),
3             min_length = min(song_length), max_length = max(song_length),
4             min_tempo = min(tempo))
```

# A tibble: 1 × 5

|   | first_release | last_release | min_length | max_length | min_tempo |
|---|---------------|--------------|------------|------------|-----------|
|   | <date>        | <date>       | <dbl>      | <dbl>      | <dbl>     |
| 1 | 1942-01-01    | 2029-04-06   | 0.0167     | 29.8       | 0         |

Releases in **2029**, a **1-second** song, a tempo of **0 bpm**. Some values are clearly errors, others merely unusual (intros, medleys) — **inspect individually** before deciding.

## Check 4: what is a row, really?

```
1 music_data |> summarise(n_rows = n(), n_distinct_isrc = n_distinct(isrc))
```

```
# A tibble: 1 × 2  
  n_rows n_distinct_isrc  
  <int>      <int>  
1  66796          56945
```

The ISRC should identify a recording uniquely — yet there are **9,851 more rows than ISRCs**, and the rows differ in streams and chart weeks. Versions? Periods? Errors? Without documentation we cannot tell — but “one row = one recording” **does not hold**.

## Check 5: outliers are not always errors

```
1 music_data |> arrange(desc(streams)) |> select(artist, track, streams) |> head(4)
```

```
# A tibble: 4 × 3
```

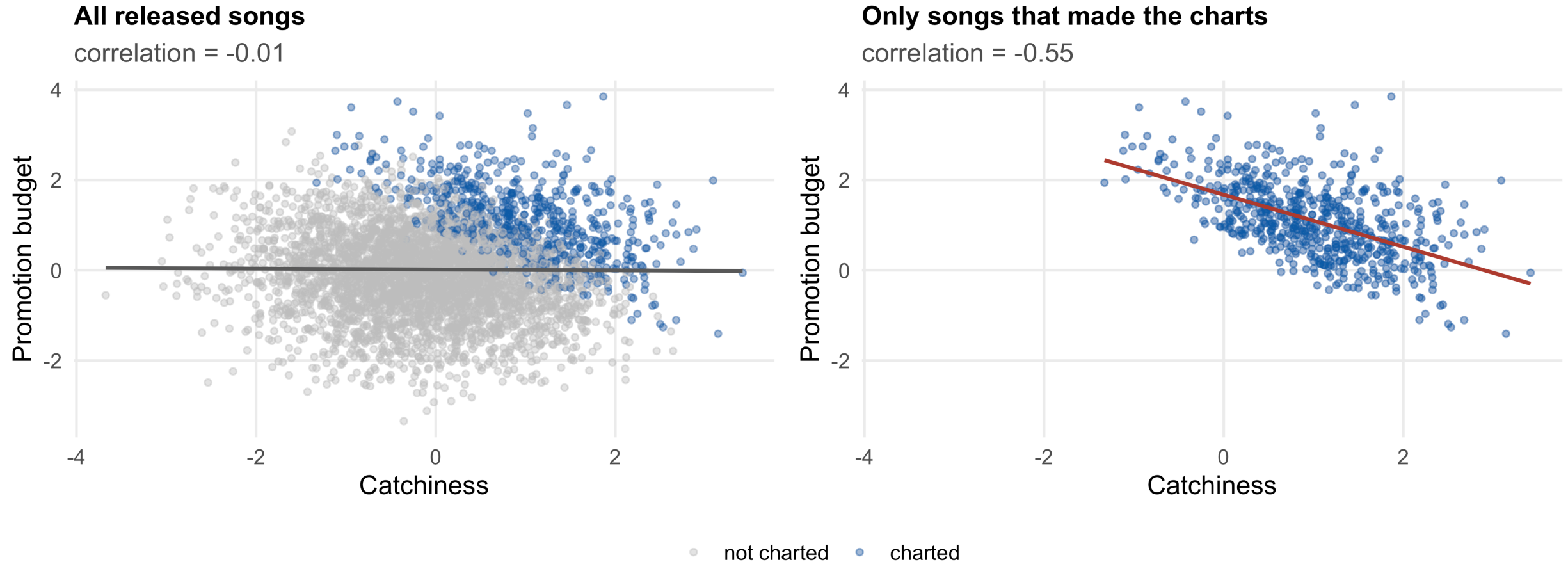
|   | artist        | track             | streams    |
|---|---------------|-------------------|------------|
|   | <chr>         | <chr>             | <dbl>      |
| 1 | Ed Sheeran    | Shape of You      | 2165692479 |
| 2 | Tones and I   | Dance Monkey      | 1909947000 |
| 3 | Billie Eilish | bad guy           | 1459149566 |
| 4 | Lewis Capaldi | Someone You Loved | 1419867299 |

In markets for creative products, **a few hits account for a large share of consumption**. Here, the extremes are the story — check them, don't delete them.

**Document, don't silently delete:** corrections in code, explicit flags instead of overwriting, transparent reporting of what changed.



# How did the data come to be?



Only **charted** tracks are in the data. Catchiness and promotion are unrelated among all songs (left) — but **negatively related among the survivors** (right). Created purely by the selection.

# Survivorship is everywhere in marketing data

- Successful **products** only (launch data of products still on the market)
- **Customers who did not churn** (loyalty-program data)
- **Companies that still exist** (“what do successful firms do?”)
- **Survey respondents** — who chooses to answer is itself a choice (→ your group project!)

Add one question to every descriptive checklist: **who is *not* in my data — and why?** Relationships among the survivors can differ from those in the population you care about (*Berkson's paradox*).

## 2.6 Summarizing data

# Categorical variables: frequencies

```
1 music_data |>
2   count(genre, sort = TRUE) |>
3   mutate(share = n / sum(n)) |>
4   head(5)
```

# A tibble: 5 × 3

|   | genre<br><fct> | n<br><int> | share<br><dbl> |
|---|----------------|------------|----------------|
| 1 | Pop            | 30069      | 0.450          |
| 2 | HipHop/Rap     | 21131      | 0.316          |
| 3 | other          | 5228       | 0.0783         |
| 4 | Rock           | 4214       | 0.0631         |
| 5 | Electro/Dance  | 2703       | 0.0405         |

# Conditional frequencies

```
1 music_data |>
2   group_by(genre) |>
3   summarise(n = n(), share_top10 = mean(top10 == "yes")) |>
4   arrange(desc(share_top10)) |>
5   mutate(share_top10 = scales::percent(share_top10, accuracy = 0.1))
```

# A tibble: 11 × 3

|    | genre         | n     | share_top10 |
|----|---------------|-------|-------------|
|    | <fct>         | <int> | <chr>       |
| 1  | Country       | 504   | 26.4%       |
| 2  | other         | 5228  | 19.4%       |
| 3  | R&B           | 1881  | 17.2%       |
| 4  | Electro/Dance | 2703  | 14.2%       |
| 5  | Soundtrack    | 326   | 13.8%       |
| 6  | Reggae        | 121   | 11.6%       |
| 7  | HipHop/Rap    | 21131 | 9.7%        |
| 8  | Pop           | 30069 | 7.9%        |
| 9  | Rock          | 4214  | 7.4%        |
| 10 | German Folk   | 539   | 5.2%        |
| 11 | Classics/Jazz | 80    | 1.2%        |

Beware of **small groups**: a share computed from 80 tracks is far less reliable than one from 30,000 — made precise in Module 3.

# Mean or median?

```
1 music_data |>
2   summarise(mean = mean(streams), median = median(streams),
3             q25 = quantile(streams, 0.25), q75 = quantile(streams, 0.75))
```

# A tibble: 1 × 4

|   | mean     | median  | q25   | q75      |
|---|----------|---------|-------|----------|
|   | <dbl>    | <dbl>   | <dbl> | <dbl>    |
| 1 | 7314674. | 333336. | 55760 | 2181086. |

“The average track has **7.3 million** streams.” — Is this a good description of the typical track?

No: a few mega-hits pull the mean up — the distribution is **right-skewed**. The **median** (~333,000) describes the typical track. Mean far above median → a few extremes dominate the average (sales, revenue, followers, streams).

# Location, dispersion, shape — in one call

```
1 library(psych)
2 music_data |>
3   select(streams, danceability, energy, valence, song_length) |>
4   describe() |>
5   select(n, mean, sd, median, min, max, skew)
```

|              | n     | mean       | sd          | median    | min     | max           | skew  |
|--------------|-------|------------|-------------|-----------|---------|---------------|-------|
| streams      | 66796 | 7314673.94 | 39956263.68 | 333335.50 | 1003.00 | 2165692479.00 | 16.74 |
| danceability | 66796 | 66.32      | 14.71       | 68.00     | 0.00    | 98.30         | -0.53 |
| energy       | 66796 | 64.37      | 17.52       | 66.10     | 0.00    | 100.00        | -0.54 |
| valence      | 66796 | 50.42      | 22.26       | 50.00     | 0.00    | 99.00         | 0.08  |
| song_length  | 66796 | 3.43       | 0.86        | 3.34      | 0.02    | 29.78         | 3.36  |

- **Location:** mean, median
- **Dispersion:** SD, range, IQR, coefficient of variation (SD / mean)
- **Shape:** skewness — `streams`  $\approx 17$ : extremely right-skewed

**z-scores** express relative position:  $z = (x - \bar{x})/s \rightarrow$  “1.5 SDs above the average track”. Their link to probabilities: Module 3.

## 2.7 Visualizing data

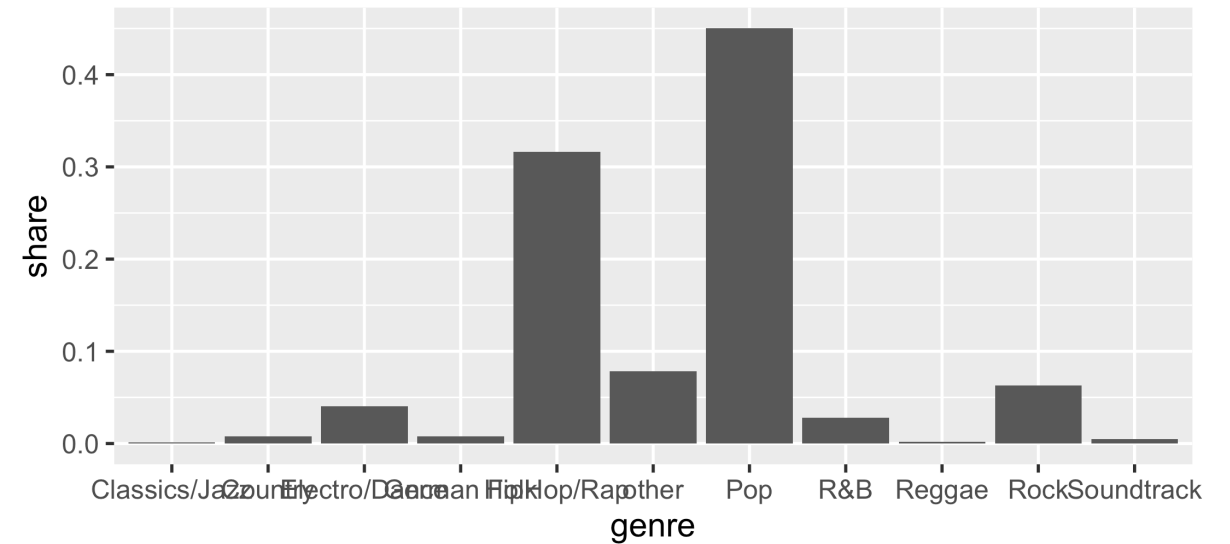


# The grammar of graphics

Every `ggplot2` chart is built from the same components, **layer** by **layer** with `+`:

1. **Data** — `ggplot(music_data, ...)`
2. **Aesthetic mappings** — `aes(x = ..., y = ..., color = ...)`
3. **Geoms** — `geom_col()`, `geom_point()`, `geom_boxplot()`, ...
4. **Labels, scales, themes** — `labs()`, `scale_...()`, `theme_...()`

```
1 genre_shares <- music_data |>
2   count(genre) |> mutate(share = n / sum(n))
3 ggplot(genre_shares, aes(x = genre, y = share)) +
4   geom_col()
```

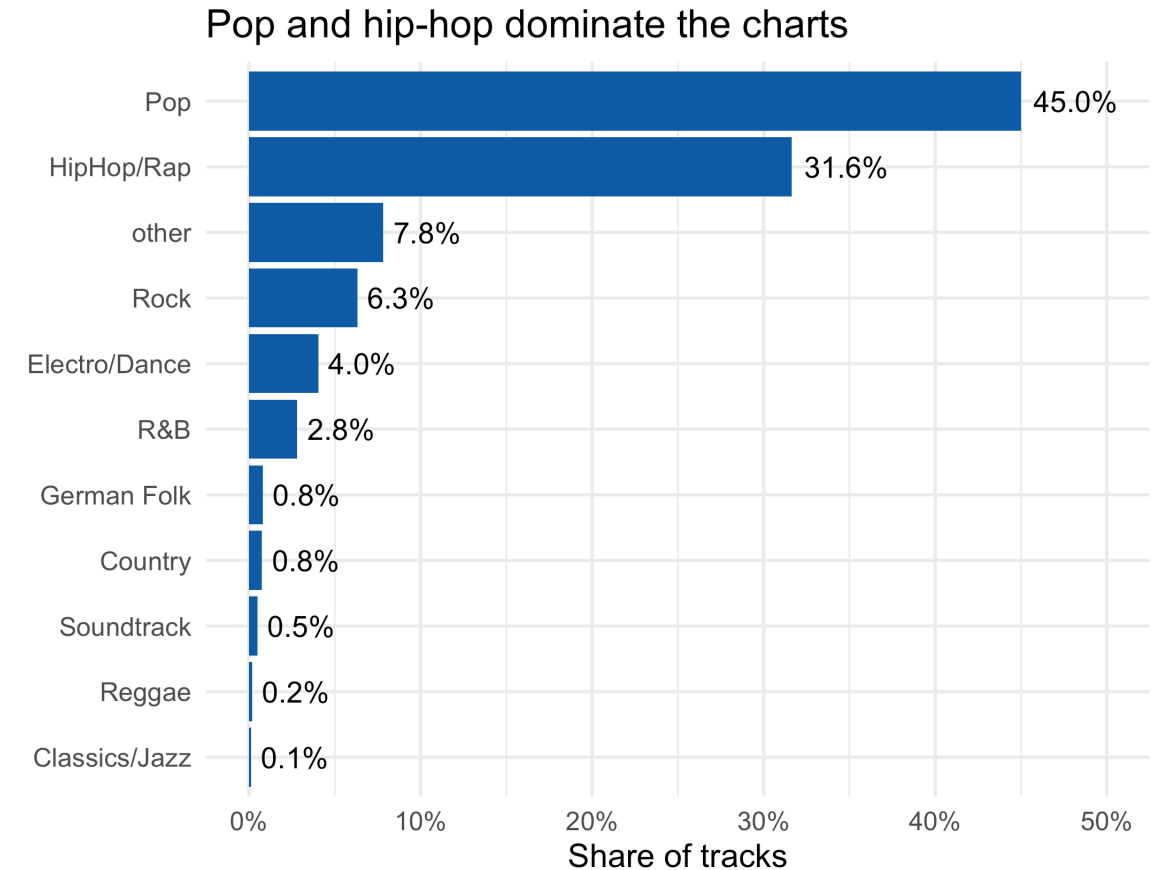


# From default to useful

```

1 ggplot(genre_shares,
2       aes(x = fct_reorder(genre, share), y = share)) +
3   geom_col(fill = "#0f5da8") +
4   geom_text(aes(label = scales::percent(share, accuracy = 0.1)),
5             hjust = -0.15, size = 3.5) +
6   coord_flip() +
7   scale_y_continuous(labels = scales::percent,
8                     limits = c(0, 0.5)) +
9   labs(x = NULL, y = "Share of tracks",
10        title = "Pop and hip-hop dominate the charts") +
11   theme_minimal()

```



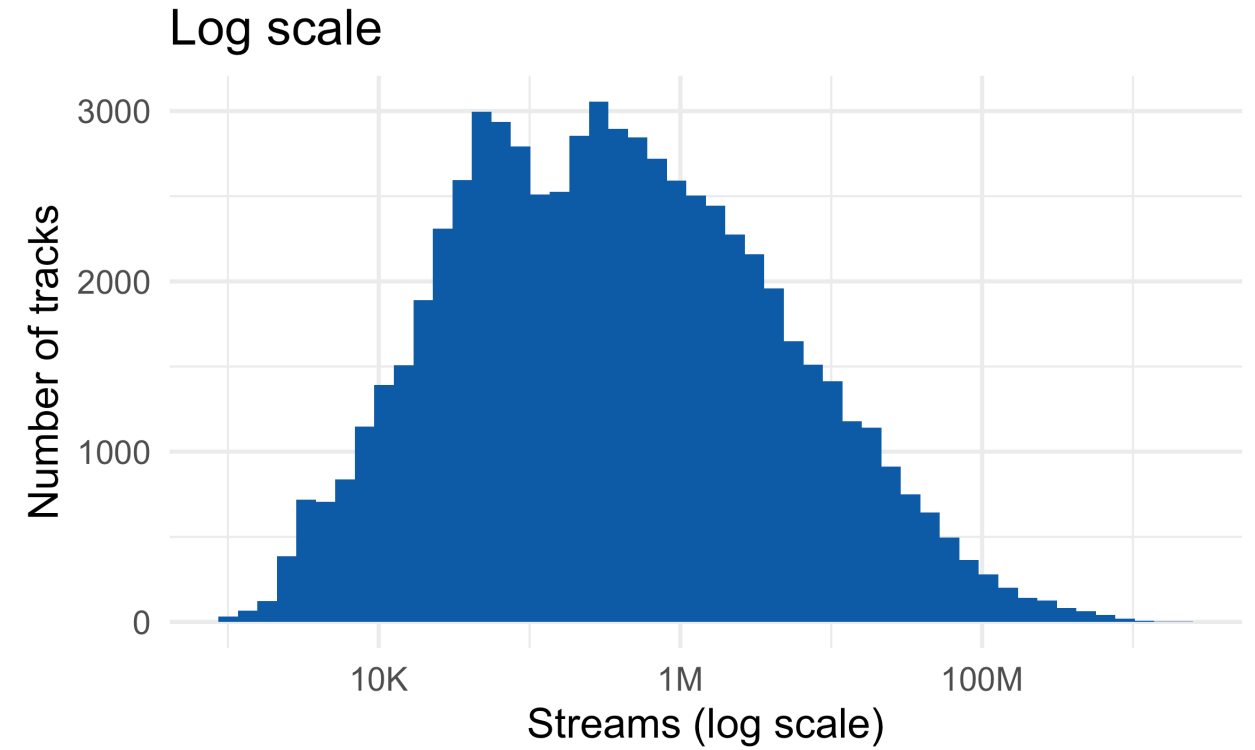
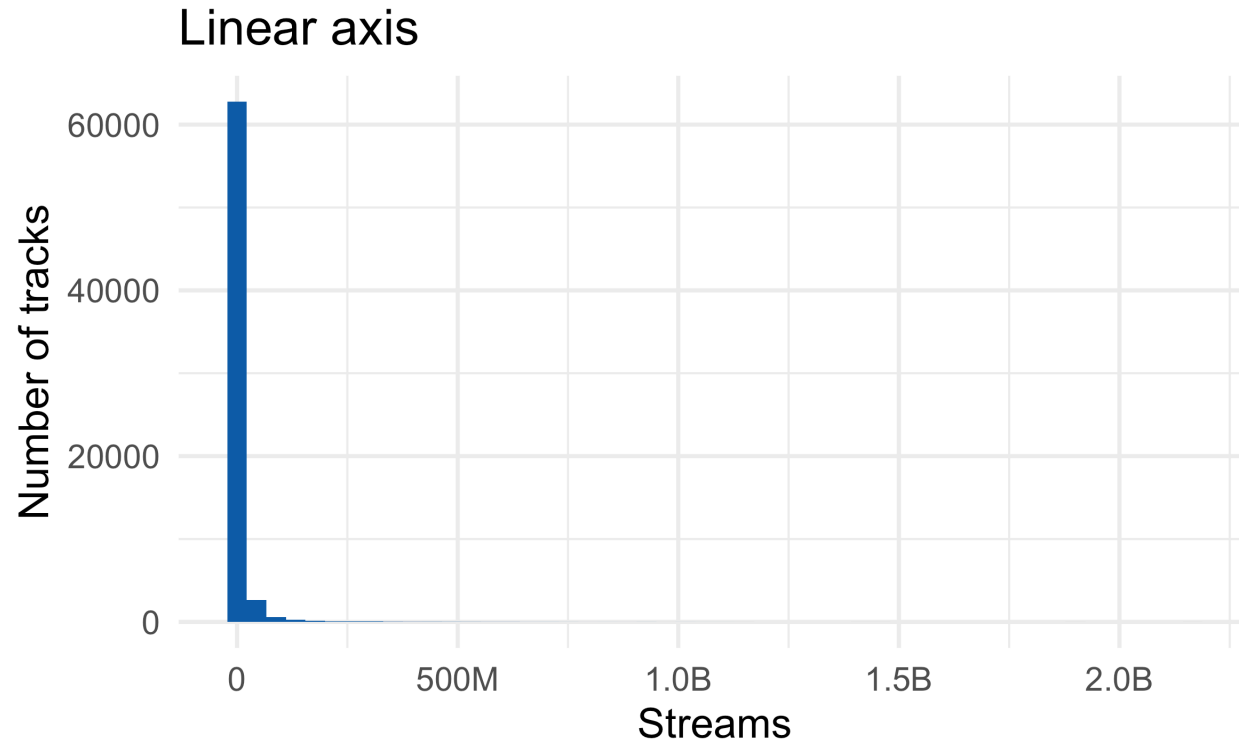
Sorted bars · readable labels · percent axis · a title that **states the finding**, not the topic.

# Which chart for which variables?

| To show ...      | Variables                 | Chart                            | geom                                                                  |
|------------------|---------------------------|----------------------------------|-----------------------------------------------------------------------|
| One distribution | categorical               | Bar chart                        | <code>geom_bar() / geom_col()</code>                                  |
|                  | continuous                | Histogram, density, boxplot      | <code>geom_histogram(),<br/>geom_density(),<br/>geom_boxplot()</code> |
| Groups compared  | categorical × continuous  | Boxplots, jittered points        | <code>geom_boxplot(),<br/>geom_jitter()</code>                        |
| A relationship   | continuous × continuous   | Scatter plot                     | <code>geom_point()</code>                                             |
| Two categoricals | categorical × categorical | Grouped / stacked bars of shares | <code>geom_col(position = "dodge")</code>                             |
| Change over time | time × continuous         | Line chart                       | <code>geom_line()</code>                                              |

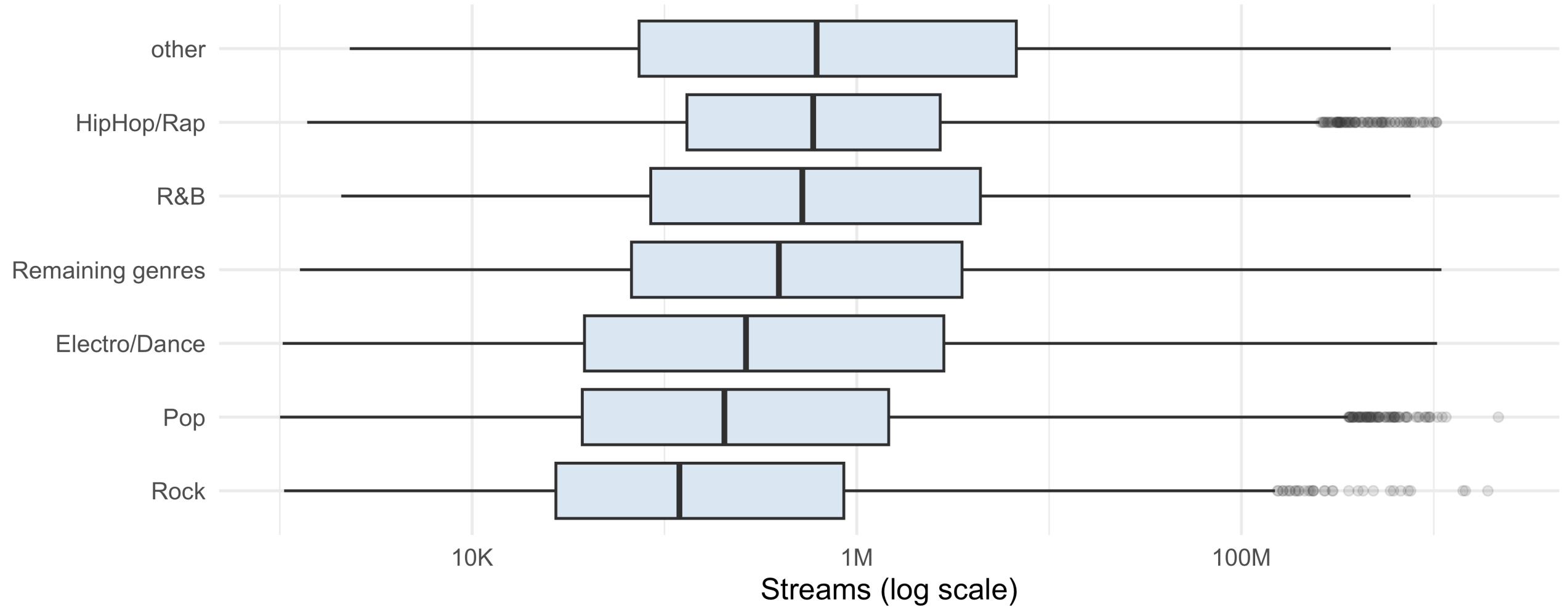
A third variable: **color** (`aes(color = ...)`) or **small multiples** (`facet_wrap(~ ...)`).

# Skewed data: the log scale

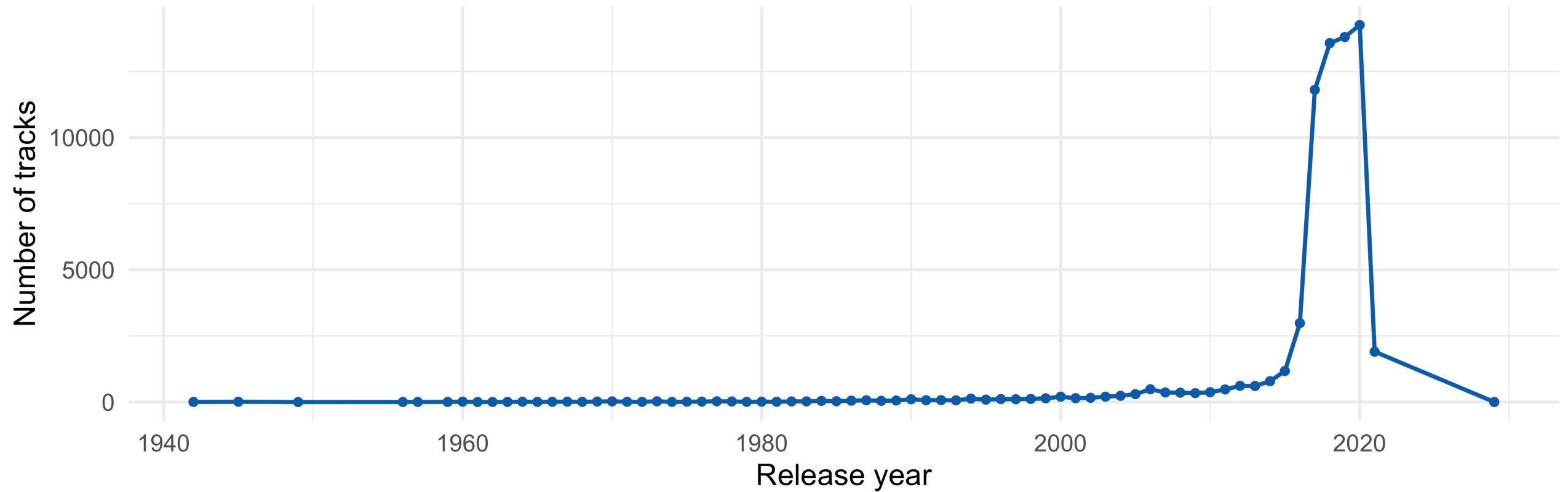


On a log scale, each step **multiplies** (10K → 100K → 1M). Makes skewed distributions readable — **always label it** in the axis title.

# Comparing groups: boxplots

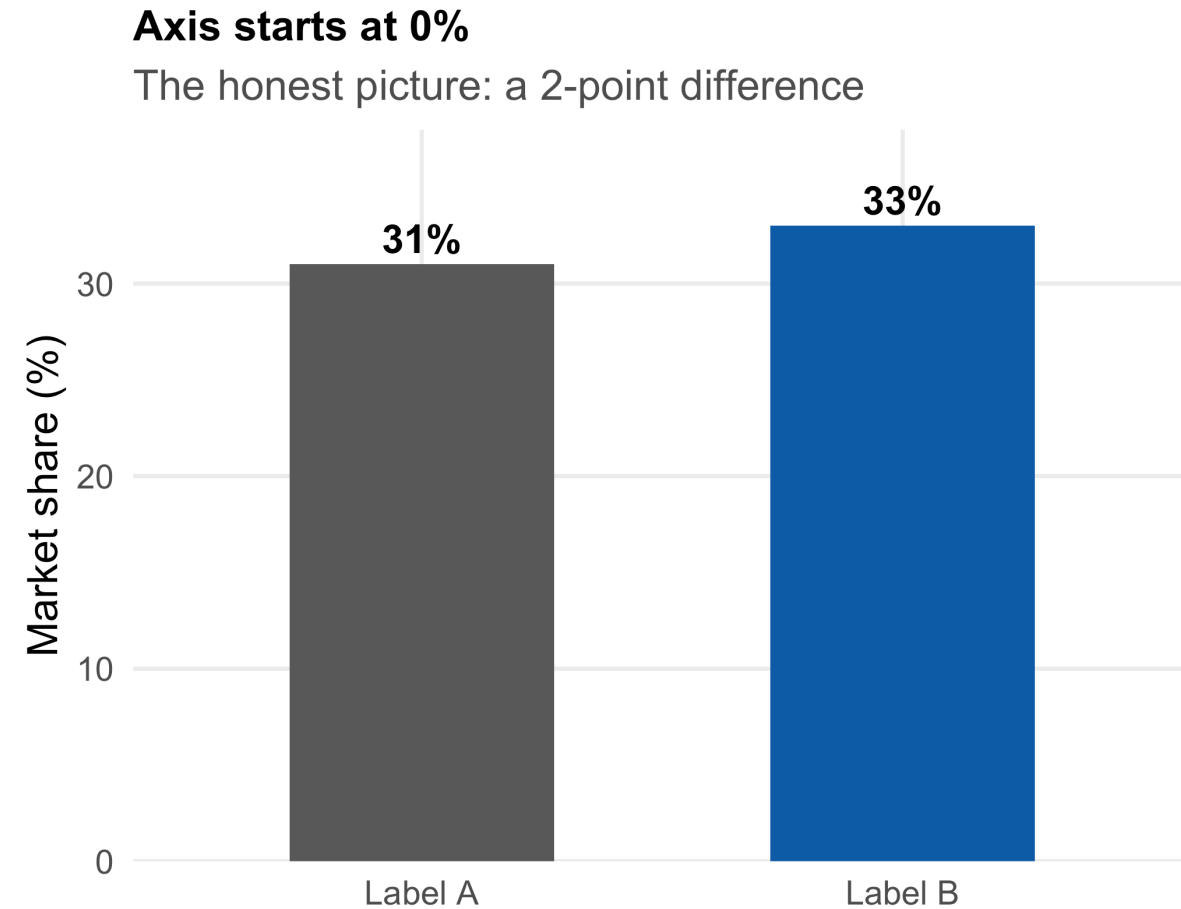
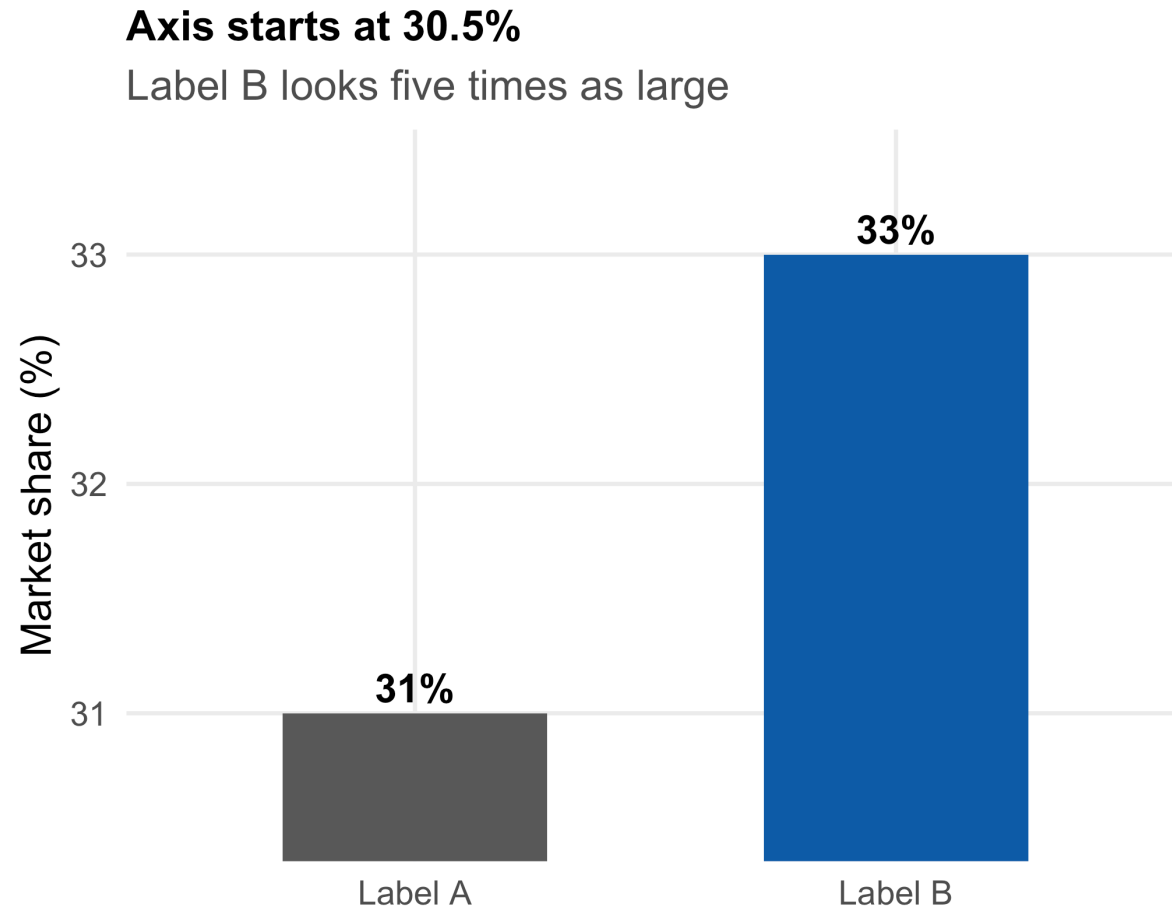


# Charts are data-quality tools, too



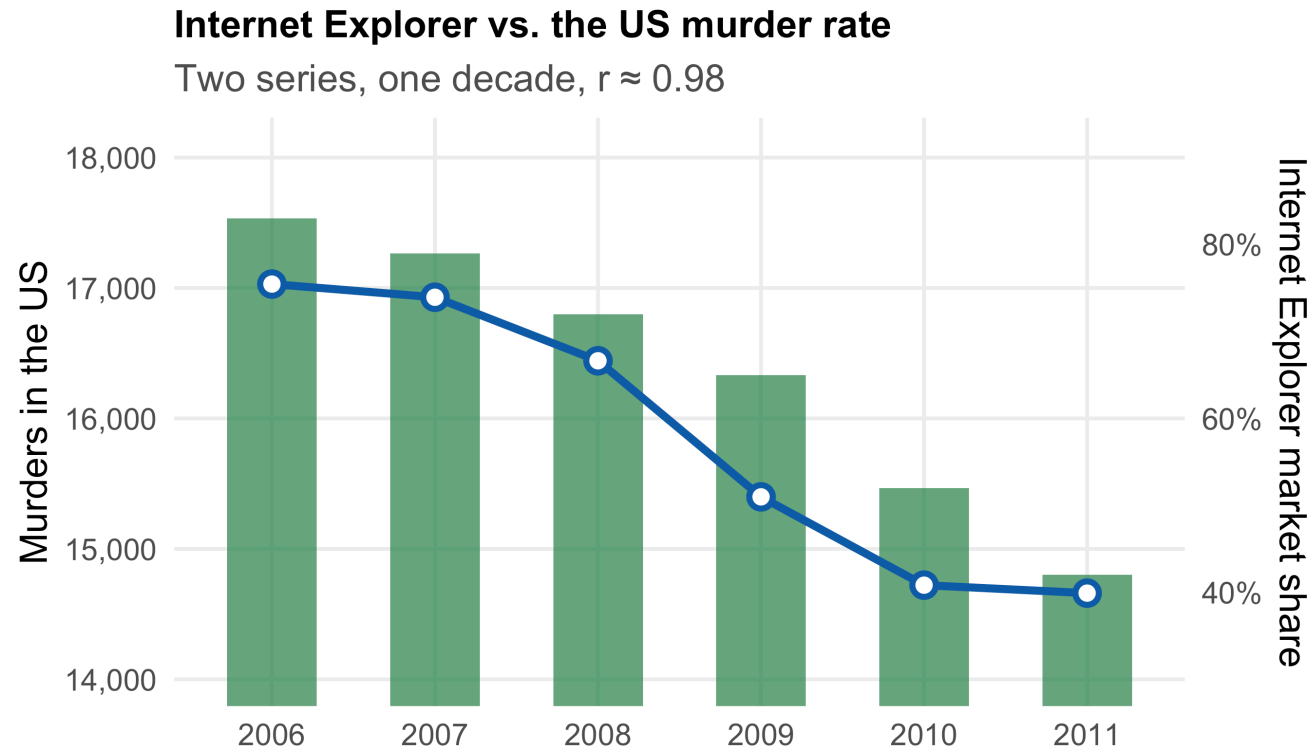
Two things in this chart should make you suspicious. **Which?**

# Honest visualization I: bars start at zero



Bars encode values by **length** — a truncated axis makes the lengths lie. (Points and lines encode by *position* and may start elsewhere — clearly labeled.)

# Honest visualization II: dual axes



Remember this one from §1.4?

Two independently chosen y-axes let you make **almost any two series look related**.  
Alternatives: index both series (first year = 100), or two charts side by side.

- Also avoid: **3D effects**, **pies** with many slices, colors without meaning, unsorted categories, axes without units



## 2.8 From description to insight

# “Do explicit tracks perform worse?”

```
1 music_data |>
2   group_by(explicit) |>
3   summarise(n = n(), median_streams = median(streams))
```

```
# A tibble: 2 × 3
  explicit      n median_streams
  <fct>      <int>          <dbl>
1 not explicit 58603      385290
2 explicit    8193       93600
```

Can we write “*explicit lyrics reduce streams*”? What would you check first?

# Looking within genres

```
1 music_data |>
2   mutate(genre = fct_lump_n(genre, n = 5, other_level = "Remaining genres")) |>
3   group_by(genre) |>
4   summarise(share_explicit      = mean(explicit == "explicit"),
5             median_not_explicit = median(streams[explicit == "not explicit"]),
6             median_explicit    = median(streams[explicit == "explicit"]))
```

# A tibble: 6 × 4

|   | genre<br><fct>   | share_explicit<br><dbl> | median_not_explicit<br><dbl> | median_explicit<br><dbl> |
|---|------------------|-------------------------|------------------------------|--------------------------|
| 1 | Electro/Dance    | 0.340                   | 494986.                      | 72608                    |
| 2 | HipHop/Rap       | 0.0515                  | 618344.                      | 226571                   |
| 3 | other            | 0.0779                  | 704424                       | 133351                   |
| 4 | Pop              | 0.155                   | 236848.                      | 87798                    |
| 5 | Rock             | 0.172                   | 159262.                      | 31128                    |
| 6 | Remaining genres | 0.112                   | 536213                       | 224444                   |

The gap persists within each large genre — but its size **and the share of explicit tracks** vary strongly. Genre, label, artist popularity, and the selection into the charts are all candidate **confounders** (§1.5).

# What we can — and cannot — write

## Descriptive ✓

*“Among the tracks in our chart data, those with explicit lyrics have a lower median number of streams than those without (about 94,000 vs. 385,000). The gap is visible within each large genre, but varies in size.”*

## Causal ✗

*“Explicit lyrics reduce streams.”*

That would require comparing the **same** tracks with and without explicit lyrics — a counterfactual the data cannot provide (§1.2).

A good descriptive finding answers: **What** was measured? **How much** (numbers, not adjectives)? **Compared to what? For whom? With which caveat?** — and uses descriptive verbs: *is higher, differs, is associated with* — not *causes, drives, increases*.

# Assignment 1: data handling & description

- **Your personal version of the data** (based on your student ID) — check, handle, summarize, visualize
- Submit as a **reproducible report** (.Rmd → HTML) via Canvas
- **Deadline:** Group A: Mon, Oct 26 · Group B: Wed, Oct 28
- **Debrief:** 3 short paper questions about *your* results at the start of session 4 (closed book)
- **10 points:** report 7 + debrief 3

Everything you need is in **this chapter**. AI assistants welcome — you must be able to **explain every line** you submit.

# Learning check

1. After `read_csv("file.csv")` on a file from a German Excel, the data frame has **one column**. Why — and what should you use instead?
2. Mean revenue per customer: **€480**, median: **€95**. What does this tell you about the distribution? Which number describes the typical customer?
3. A loyalty-program data set contains **only active customers**. Among them, those who received few discounts buy most often. Why might this pattern mislead?
4. **True or false?** (a) A variable stored as a number can be nominal. (b) Outliers should always be removed. (c) The y-axis of a bar chart should start at zero. (d) A zero in a data set always means the quantity was zero.

# Next session

## Module 3 · Uncertainty & Experiments I — *How sure can I be?*

- Why every number from a sample is uncertain — and how to quantify it
- Standard errors and confidence intervals — by simulation
- Our first experiment analysis: comparing two groups

Course book chapter 2 (consolidation reading for today) is online — with all code from these slides. **Assignment 1** is on Canvas.